

# Git Workflow Guide

A Simplified Version Control Guide for Working Directly on theOriginal Branch

Welcome to the **ASU ROAR Simulation Team**! To keep our development workflow simple, all simulation and package work resides directly within a single main branch named **theOriginal**. This guide explains how to configure Git to download *only* this branch, add or edit your ROS 2 packages, and stay synchronized with team updates directly on this branch.

## REPOSITORY DETAILS: Repository URL (SSH):

git@github.com:ASU-ROAR-Team/ROAR-26-Simulation.git

**Repository URL (HTTPS):** https://github.com/ASU-ROAR-Team/ROAR-26-Simulation.git

**Target branch:** theOriginal

## 1. Cloning Only the target Branch

To save bandwidth, speed up operations, and avoid downloading unnecessary branches, you should clone **only** the branch where all work resides (theOriginal). Navigate to your ROS 2 workspace src folder and run the clone command with the -b and --single-branch options:

```
$ cd ~/Desktop/ROAR/simulation_ws/src
$ git clone -b theOriginal --single-branch git@github.com:ASU-ROAR-Team/ROAR-26-Simulation.git
```

Once cloned, enter the directory. You are automatically set up on the correct branch:

```
$ cd ROAR-26-Simulation
$ git branch
* theOriginal
```

## 2. Adding & Editing Your ROS 2 Package

Create your package directories or copy your package files into the repository (typically under the rover / or package root folder). As you work, you can query Git to see modified and untracked files:

```
$ git status
```

To inspect exact line-by-line code edits you've made to existing tracked files in the workspace:

```
$ git diff
```

### 3. Keeping Updated (Fetch & Pull)

Because the entire team works directly on the `theOriginal`, new changes will be pushed to the remote repository regularly. To keep your local repository up to date and avoid conflicts, you **must** fetch and pull remote updates frequently (ideally before starting a session and always before staging code):

**Step 3.1: Fetch remote changes** (updates your git catalog without touching your active files):

```
$ git fetch origin
```

**Step 3.2: Pull updates into your local branch** (merges team edits into your files):

```
$ git pull origin theOriginal
```

#### **CONFLICT WARNING: Handling Merge Conflicts**

If someone else edited the same lines of the same files you are currently modifying, Git will report a merge conflict during the pull. Open the conflicted files in a code editor, locate the conflict markers (`<<<<<<<` and `=====`), decide which team member's code to keep, delete the conflict markers, and save. Once fixed, stage the resolved files.

### 4. Staging & Committing Your Package

When your package edits are functional, stage and save them locally. Ensure you do not add temporary build folders (like `build/`, `install/`, or `log/`).

Stage your new package directory to Git:

```
$ git add rover/your_package_name/
```

Or stage all modified files that are already tracked by Git:

```
$ git add -u
```

Commit your changes with a short, descriptive message explaining your work:

```
$ git commit -m "feat: added launcher and nodes to rover package"
```

### 5. Pushing Changes Back to the Remote Branch

Finally, publish your committed work to the remote repository so other team members can pull your updates:

```
$ git push origin theOriginal
```

## ASU ROAR Git Commands Quick Reference

A direct reference guide for your daily single-branch workspace workflow:

| Git Command                                                 | Description / Action                                     | When to Use                                         |
|-------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------|
| <code>git clone -b theOriginal --single-branch [URL]</code> | Clones ONLY the target branch from GitHub.               | Once, when setting up the workspace.                |
| <code>git status</code>                                     | Lists modified, staged, and untracked files.             | Frequently, to check workspace status.              |
| <code>git diff</code>                                       | Shows line-by-line differences in modifications.         | Before adding/committing to verify changes.         |
| <code>git fetch origin</code>                               | Downloads latest history changes from GitHub.            | Frequently, to check for team updates.              |
| <code>git pull origin theOriginal</code>                    | Fetches and merges remote changes into your local files. | Daily, before editing or staging files.             |
| <code>git add [path]</code>                                 | Stages specific files/folders for commit.                | When your package edits are complete.               |
| <code>git add -u</code>                                     | Stages all modified tracked files.                       | To stage all updates at once.                       |
| <code>git commit -m "[msg]"</code>                          | Saves staged changes locally with a message.             | After staging files using git add.                  |
| <code>git push origin theOriginal</code>                    | Uploads local commits directly to the remote branch.     | After committing, when your code is ready to share. |

### REMINDER: Need Help?

If you encounter merge conflicts or errors you can't resolve, contact the lead repository maintainer. Avoid using force push (`git push -f`) on theOriginal as it will overwrite team member commits.